

▼ Phần 1: Giới thiệu và cài đặt

```
# Cài đặt spaCy
!pip install -U spacy
# Tải về mô hình tiếng Anh (kích thước trung bình, có đủ thông tin cho parsing)
!python -m spacy download en_core_web_md
```

Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.20.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.1)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (25.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (2.41.4)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (0.4.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2025.11.12)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (1.3.3)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (0.1.5)
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spacy) (8.3.1)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (0.23.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (7.5.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2->spacy) (2.0.1)
Collecting en-core-web-md==3.8.0
 Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_md-3.8.0/en_core_web_md-3.8.0-py3-none-any.whl (33.5 MB)
33.5/33.5 MB 32.6 MB/s eta 0:00:00

✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_md')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

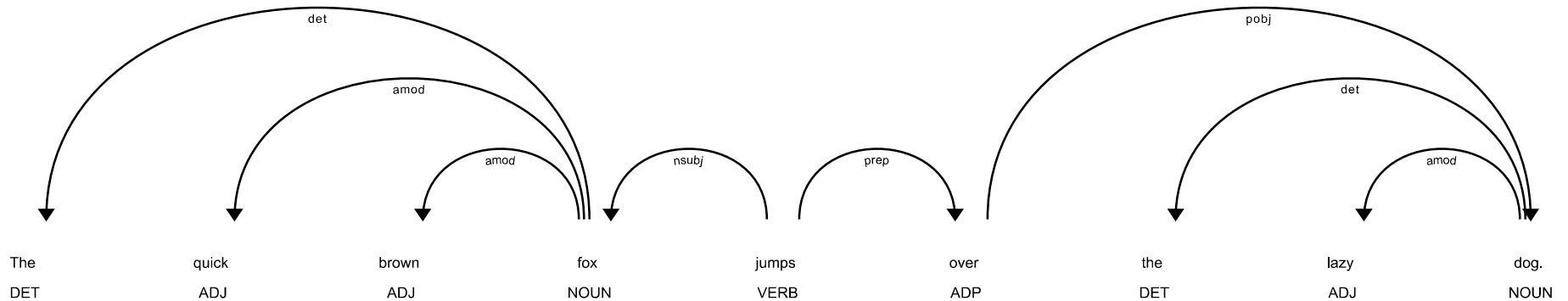
▼ Phần 2: Phân tích và trực quan hóa

```
import spacy
from spacy import displacy
# Tải mô hình tiếng Anh đã cài đặt
# Sử dụng en_core_web_md vì nó chứa các vector từ và cây cú pháp đầy đủ
nlp = spacy.load("en_core_web_md")
# Câu ví dụ
text = "The quick brown fox jumps over the lazy dog."
```

```
# Phân tích câu với pipeline của spaCy
doc = nlp(text)
```

```
# Tùy chọn để hiển thị trong trình duyệt
options = {"compact": True, "color": "blue", "font": "Source Sans Pro"}
# Khởi chạy server tại http://127.0.0.1:5000
# Bạn có thể truy cập địa chỉ này trên trình duyệt để xem cây phụ thuộc
# Nhấn Ctrl+C trong terminal để dừng server
displacy.serve(doc, style="dep")
```

```
/usr/local/lib/python3.12/dist-packages/spacy/displacy/__init__.py:108: UserWarning: [W011] It looks like you're calling displacy.serve from within a Jupyter notebook or a similar environment. This li
warnings.warn(Warnings.W011)
```



Using the 'dep' visualizer
Serving on <http://0.0.0.0:5000> ...

Shutting down server on port 5000.

Trong câu này từ "jumps" là gốc của câu

Những từ phụ thuộc của "jumps" :

- "fox": quan hệ nsubj, tức là chủ ngữ của động từ "jumps"
- "over": quan hệ prep, giới từ - mở đầu cho cụm giới từ sau đó

"fox" là head của những từ:

- "The"
- "quick"
- "brown"

▼ Phần 3: Truy cập các thành phần trong cây phụ thuộc

```
# Lấy một câu khác để phân tích
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)
# In ra thông tin của từng token
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | {'CHILDREN'}")
print("-" * 70)

for token in doc:
    # Trích xuất các thuộc tính
    children = [child.text for child in token.children]
    print(f"{'token.text':<12} | {'token.dep_':<10} | {'token.head.text':<12} | {'token.head.pos_':<8} | {'children'}")
```

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN
Apple	nsubj	looking	VERB	[]
is	aux	looking	VERB	[]
looking	ROOT	looking	VERB	['Apple', 'is', 'at']
at	prep	looking	VERB	['buying']
buying	pcomp	at	ADP	['startup']
U.K.	compound	startup	NOUN	[]
startup	dobj	buying	VERB	['U.K.', 'for']
for	prep	startup	NOUN	['billion']
\$	quantmod	billion	NUM	[]
1	compound	billion	NUM	[]
billion	pobj	for	ADP	['\$', '1']

▼ Phần 4: Duyệt cây phụ thuộc để trích xuất thông tin

▼ 4.1 Bài toán tìm chủ ngữ và tân ngữ của một động từ

```
text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)
for token in doc:
    # Chỉ tìm các động từ
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""
        obj = ""
        # Tìm chủ ngữ (nsubj) và tân ngữ (dobj) trong các con của động từ
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text
        if subject and obj:
            print(f"Found Triplet: ({subject}, {verb}, {obj})")
```

```
Found Triplet: (cat, chased, mouse)
Found Triplet: (cat, chased, mouse)
Found Triplet: (cat, chased, mouse)
Found Triplet: (dog, watched, them)
Found Triplet: (dog, watched, them)
```

▼ 4.2: Bài toán tìm các từ bổ nghĩa cho một danh từ

```
text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)
for token in doc:
```

```
# Chỉ tìm các danh từ
if token.pos_ == "NOUN":
    adjectives = []
    # Tìm các tính từ bổ nghĩa (amod) trong các con của danh từ
    for child in token.children:
        if child.dep_ == "amod":
            adjectives.append(child.text)
    if adjectives:
        print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ: {adjectives}")
```

Danh từ 'cat' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']
Danh từ 'mat' được bổ nghĩa bởi các tính từ: ['warm']

▼ Phần 5: Bài tập tự luyện

▼ Bài 1: Tìm động từ chính trong câu

```
def find_main_verb(doc):
    for token in doc:
        if token.dep_ == "ROOT" and token.pos_ == "VERB":
            return token
    return None

text1 = "The cat chased the mouse and the dog watched them."
doc1 = nlp(text1)
main_verb1 = find_main_verb(doc1)
if main_verb1:
    print(f"Động từ chính trong câu '{text1}' là: {main_verb1.text}")

text2 = "The quick brown fox jumps over the lazy dog."
doc2 = nlp(text2)
main_verb2 = find_main_verb(doc2)
if main_verb2:
    print(f"Động từ chính trong câu '{text2}' là: {main_verb2.text}")

text3 = "Apple is looking at buying U.K. startup for $1 billion"
doc3 = nlp(text3)
main_verb3 = find_main_verb(doc3)
if main_verb3:
    print(f"Động từ chính trong câu '{text3}' là: {main_verb3.text}")
```

Động từ chính trong câu 'The cat chased the mouse and the dog watched them.' là: chased
Động từ chính trong câu 'The quick brown fox jumps over the lazy dog.' là: jumps
Động từ chính trong câu 'Apple is looking at buying U.K. startup for \$1 billion' là: looking

▼ Bài 2: Trích xuất các cụm danh từ (Noun chunks)

```
def extract_noun_chunks(doc):
    noun_chunks = []
    # Sử dụng một tập hợp để theo dõi các chỉ số token đã được bao phủ bởi một cụm,
    # giúp tránh các cụm chồng chéo hoặc trùng lặp.
    covered_indices = set()

    for token in doc:
        # Chỉ xem xét các token là danh từ hoặc danh từ riêng và chưa được bao phủ
        if token.pos_ in ["NOUN", "PROPN"] and token.i not in covered_indices:
            # Khởi tạo danh sách các token trong cụm với chính danh từ đó
```

```

current_chunk_tokens = [token]

# Tìm các từ bổ nghĩa đứng trước (pre-modifiers) là con của danh từ này
for child in token.children:
    # Các loại phụ thuộc thường là từ bổ nghĩa của danh từ
    if child.dep_ in ["det", "amod", "compound", "nummod", "poss", "neg"]:
        current_chunk_tokens.append(child)

# Sắp xếp các token theo chỉ số của chúng để có đúng thứ tự trong câu
current_chunk_tokens.sort(key=lambda t: t.i)

# Nếu có token trong cụm, tạo một đối tượng Span và thêm vào danh sách kết quả
if current_chunk_tokens:
    # Span bắt đầu từ chỉ số nhỏ nhất và kết thúc ở chỉ số lớn nhất (bao gồm)
    chunk_span = doc[current_chunk_tokens[0].i : current_chunk_tokens[-1].i + 1]
    noun_chunks.append(chunk_span.text)

# Đánh dấu tất cả các token trong cụm này là đã được bao phủ
for i in range(current_chunk_tokens[0].i, current_chunk_tokens[-1].i + 1):
    covered_indices.add(i)

return noun_chunks

print(f"Cụm danh từ trong câu '{text1}':\n {extract_noun_chunks(doc1)}")

print(f"Cụm danh từ trong câu '{text2}':\n {extract_noun_chunks(doc2)}")

print(f"Cụm danh từ trong câu '{text3}':\n {extract_noun_chunks(doc3)}")

```

```

Cụm danh từ trong câu 'The cat chased the mouse and the dog watched them.':
['The cat', 'the mouse', 'the dog']
Cụm danh từ trong câu 'The quick brown fox jumps over the lazy dog.':
['The quick brown fox', 'the lazy dog']
Cụm danh từ trong câu 'Apple is looking at buying U.K. startup for $1 billion':
['Apple', 'U.K.', 'U.K. startup']

```

▼ Bài 3: Tìm đường đi ngắn nhất trong cây

```

def get_path_to_root(token):
    path = []
    current_token = token
    while current_token != current_token.head:
        path.append(current_token)
        current_token = current_token.head
    path.append(current_token) # Thêm ROOT token vào cuối
    return path

# Ví dụ
text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)

# Token 'fox'
fox_token = doc[3]
path_to_root_fox = get_path_to_root(fox_token)
print(f"Đường đi từ '{fox_token.text}' đến ROOT: {[t.text for t in path_to_root_fox]}")

# token 'lazy'
lazy_token = doc[7]
path_to_root_lazy = get_path_to_root(lazy_token)
print(f"Đường đi từ '{lazy_token.text}' đến ROOT: {[t.text for t in path_to_root_lazy]}")

```

```
# token 'jumps' (là ROOT trong câu này)
jumps_token = doc[4]
path_to_root_jumps = get_path_to_root(jumps_token)
print(f"Đường đi từ '{jumps_token.text}' đến ROOT: {[t.text for t in path_to_root_jumps]}")
```

```
Đường đi từ 'fox' đến ROOT: ['fox', 'jumps']
Đường đi từ 'lazy' đến ROOT: ['lazy', 'dog', 'over', 'jumps']
Đường đi từ 'jumps' đến ROOT: ['jumps']
```